

UNIVERSITY COLLEGE LONDON

EXAMINATION FOR INTERNAL STUDENTS

MODULE CODE : **COMP0008**

ASSESSMENT PATTERN: **COMP0008A5UG 001**

MODULE NAME : **COMP0008 - Computer Architecture and Concurrency**

LEVEL : **Undergraduate**

DATE: : **28-Apr-2025**

TIME : **10:00**

DURATION : **3 Hours**

This paper is suitable for candidates who attended classes for this module in the following academic year(s):

Year
2024-25

EXAMINATION PAPER CANNOT BE REMOVED FROM THE EXAM HALL. PLACE EXAM PAPER AND ALL COMPLETED SCRIPTS INSIDE THE EXAMINATION ENVELOPE

Hall instructions	N/A
Additional materials	None
Standard Calculators	No
Non-Standard Calculators	No

TURN OVER

Computer Architecture and Concurrency, COMP0008 (A5U)

Main Summer Assessment 2024/25

Suitable for Cohorts: 2024/25, 2023/24, 2022/23

This assessment is structured in two parts. Each part includes 3 questions; each question consists of 2-5 subquestions. Answer ALL the questions in your answer booklet.

For each answer you provide, **clearly indicate the corresponding question**; otherwise, the teaching staff may be unable to mark some of your answers, and hence to assign the corresponding marks. Namely, we suggest that you prepend each of your answers with the identifier of the question your answer refers to:

Q1a. <your answer>

Q1b. <your answer>

...

Marks for all questions and subquestions are reported in square brackets. They are indicated by **two values**. The first value corresponds to the marks you will be awarded for a correct answer, and the second value corresponds to the marks you will be given in the case of no answer.

For example, [(+5,+2) marks] next to a question specifies that a correct answer to that question is worth 5 marks, and no answer is worth 2 marks.

Partial marks may be given for partially correct, incomplete, vague/confused or unnecessarily long answers to open questions (i.e., *not* multiple-choice ones). However, wrong answers to any question will be given zero marks, so **DO NOT GUESS**.

To achieve full marks, always aim to provide **complete answers that are brief and to the point**. Additional instructions on expected length and format of your answers are provided at the beginning of each question.

Part ONE: Computer Architecture

Question 1. In this question, we will look into integer representations. For each subquestion, **exactly one option is correct**; do not mark multiple options. No justification is required.

Q1a. Suppose that x, y are two unsigned 32-bit integers and a, b are two signed 32-bit integers such that the representation of x is identical to that of a and the representation of y is identical to that of b . For example, $x = a = 1$ (both represented as $0x1$), $y = 2^{32} - 1$, $b = -1$ (both represented as $0xFFFFFFFF$) satisfy the criteria.

[(+5,+1.5) marks]

- A) The same hardware that checks whether $x > y$ can be used as is to determine if $a > b$.
- B) The same hardware that computes $x + y$ can always compute if $a + b$ with correct results, assuming that $x + y < 2^{32}$.
- C) The same hardware that computes $x - y$ can always compute $a - b$ with correct results, assuming that $x \geq 2^{32} - 2^{24}$ and $y \leq 2^{20}$.
- D) The same hardware that computes $x - y$ can always compute $a - b$ with correct results, assuming that $x \geq 2^{24}$ and $y = 2^{16}$.

Q1b. Suppose that $z = x \oplus y = a \oplus b$ is the **unsigned integer** whose representation is the bitwise-xor of the representations of x, y and a, b . E.g., if $x = a = 1$, $y = 2^{32} - 1$, $b = -1$ then $z = 0xFFFFFFFFE$. In the following, $\times$ stands for multiplication.

[(+5,+1.5) marks]

- A) $z \geq 2^{31}$ is only possible if $a \times b < 0$.
- B) If $a < 0$ and $b > 0$ then $z \times x > 2^{60}$.
- C) If $a < 0$ then $z \times x > 2^{30}$.
- D) If $a, b > 0$ then $z \times x < 2^{60}$.

[Total for Question 1: 10 marks]

Question 2. For each of the following, answer with **True or False**. No justification is needed.

Q2a. The endianness determines the order of bytes inside registers in CPUs such as MIPS.

[(+2,+1) marks]

Q2b. According to the convention, when writing a MIPS function code that does not call any other function, we are allowed to use the temporary registers (e.g., \$t0) without restoring their values.

[(+2,+1) marks]

Q2c. According to the MIPS convention, we may assume that after the pseudo instruction `li $s1, 100` the value of temporary registers (e.g., \$t0) is unchanged.

[(+2,+1) marks]

Q2d. According to the MIPS convention, we may assume that after the pseudo instruction `li $s1, 100` the value of kernel registers (e.g., \$k0) is not changed.

[(+2,+1) marks]

As a reminder, the UNICODE standard currently has about 155,000 different characters.

Q2e. The UTF8 encoding of any current character is, at most, three bytes long.

[(+2,+1) marks]

[Total for Question 2: 10 marks]

Question 3. In this question, each answer should be **no longer than two lines and no more than 40 words**.

Q3a. Consider the following sequence of MIPS instructions. As a reminder, `sra` is shift-right-arithmetic, and extends the sign bit when shifting.

```
sra $t1, $t0, 31
xor $t0, $t0, $t1
sub $t0, $t0, $t1
```

Explain the functionality of the following MIPS code. What's the resulting `$t0` as a function of the starting `$t0` value?

Hint: consider the case of negative and non-negative `$t0` values separately and recall that if x is a 2's complement representation of a number, then we can compute the representation of $-x$ by flipping the bits in x and adding 1.

[(+10,+2) marks]

Q3b. Explain the benefits of using conditional assignment operations (such as `set-less-than`, `slt` in MIPS) compared to using branches in an assembly code.

[(+10,+2) marks]

Q3c. Explain the benefits of reserving registers `$k0`, `$k1` for the kernel usage.

[(+10,+2) marks]

[Total for Question 3: 30 marks]

Part TWO: Concurrency

Question 4. Answer the following subquestions by writing **no more than two-three sentences per question**.

Q4a. Describe the role of hardware caches in modern CPUs: why are they used? what makes them effective?

[(+4,0) marks]

Q4b. Consider now a fictitious hardware architecture with a single processor and a single cache. In this architecture, words are aligned to multiples of four bytes, and memory addresses are 13-bit long. The cache has 16 sets, is 2-way associative, and stores two words of data (i.e., four bytes) per memory block.

Remember that when processed in caches, each memory address is interpreted as the concatenation of tag bits with set bits and byte offset ones.

i. How big in bytes is the above cache? Show your reasoning.

[(+3,0) marks]

ii. Describe the operations occurring in the cache and main memory when the processor has to load a word from the memory address 1001011100101.

[(+4,0) marks]

Q4c. Describe the implications of hardware caches on concurrent software in multi-core architectures. Can caches be responsible for any specific behaviours of concurrent programs? If so, which ones? Motivate your answer.

[(+3,0) marks]

[Total marks for Question 4: 14 marks]

Question 5. Consider the following code.

```
01: import java.util.*;

02: public class PointMover {
03:     private int x = 0, y = 0;
04:     private Object internalLock = new Object();
05:
06:     public synchronized void place (int x1, int y1) {
07:         x = x1;
08:         y = y1;
09:     }

10:     public synchronized void moveClose(int distance) {
11:         int addend = distance / 2;
12:         x += addend;
13:         y += addend;
14:     }

15:     public void moveFar(int distance) {
16:         synchronized(internalLock) {
17:             x += distance;
18:             y += distance;
19:         }
20:     }

21:     public synchronized String getPosition() {
22:         return "(" + x + "," + y + ")";
23:     }
24: }

25: public class Type1 extends Thread {
26:     private PointMover mover;

27:     public Type1(PointMover m) { mover = m; }

28:     public void run() {
29:         mover.place(10,10);
30:         mover.moveClose(4);
31:     }
32: }

33: public class Type2 extends Thread {
34:     private PointMover mover;

35:     public Type2(PointMover m) { mover = m; }
```

```

36: public void run() {
37:     mover.moveFar(2);
38: }
39: }

40: public class MoverApp {
41:     private static int numType1 = 0;
42:     private static int numType2 = 0;

43:     public static void main(String[] args)
44:         throws InterruptedException {
45:         for (String arg: args) {
46:             if (arg.startsWith("--num-t1=")) {
47:                 numType1 = Integer.parseInt(arg.split("=")[1]);
48:             }
49:             else if (arg.startsWith("--num-t2=")) {
50:                 numType2 = Integer.parseInt(arg.split("=")[1]);
51:             }
52:         }

53:         PointMover mover = new PointMover();
54:         HashSet<Thread> allThreads = new HashSet<Thread>();
55:         for (int i = 0; i < numType1; i++) {
56:             allThreads.add(new Type1(mover));
57:         }
58:         for (int i = 0; i < numType2; i++) {
59:             allThreads.add(new Type2(mover));
60:         }
61:         for (Thread t: allThreads) {
62:             t.start();
63:         }
64:         for (Thread t: allThreads) {
65:             t.join();
66:         }
67:         System.out.println(mover.getPosition());
68:     }
69: }

```

Suppose that we open a new Command Line Interface (CLI) such as a bash terminal in Linux or MacOS, and compile the code, as shown in lectures.

Answer the following subquestions by writing **no more than two-three sentences per question**.

Q5a. Describe in your own words the semantics of `synchronized(internalLock)` and its consequences for threads trying to concurrently run methods on the same `PointMover` object.

[(+8,0) marks]

Q5b. Does Java allow the instructions at lines 29 and 30 to be re-ordered at runtime? Motivate your answer.

[(+2,0) marks]

Q5c. We run `java MoverApp --num-t1=2` in a CLI. This starts two `Type1` threads. Report ALL the values that can be printed in output. Properly motivate your answer.

[(+4,0) marks]

Q5d. We now run `java MoverApp --num-t1=1 --num-t2=1` in our CLI. Doing so starts one `Type1` thread A, and one `Type2` thread B.

Will the set of values that can be printed in output differ from when we ran `java MoverApp --num-t1=2`?

If it does, describe an interleaving leading to a different output and explain why that interleaving can occur. Otherwise, provide the precise technical reasons why the set of possibly printed values remains the same.

[(+3,0) marks]

Q5e. We finally open another CLI. Then, we run `java MoverApp --num-t1=1` on one CLI and `java MoverApp --num-t2=1` on the other CLI.

Would doing so lead to the same interactions between the created threads as running `java MoverApp --num-t1=1 --num-t2=1` on either CLI?

Motivate your answer, supporting it with precise technical motivations.

[(+3,0) marks]

[Total for Question 5: 20 marks]

Question 6. Consider the following code.

```
01: public class Reader extends Thread {
02:     ReadWriteController ctrl;
03:     String id;

04:     Reader(ReadWriteController controller, String name) {
05:         ctrl = controller;
06:         id = name;
07:     }

08:     public void run() {
09:         try {
10:             while(true) {
11:                 ctrl.acquireRead();
12:                 Thread.sleep(1000);
13:                 ctrl.releaseRead();
14:             }
15:         } catch (InterruptedException e){}
16:     }
17: }

18: public class Writer extends Thread {
19:     ReadWriteController ctrl;
20:     String id;

21:     Writer(ReadWriteController controller, String name) {
22:         ctrl = controller;
23:         id = name;
24:     }

25:     public void run() {
26:         try {
27:             while(true) {
28:                 ctrl.acquireWrite();
29:                 Thread.sleep(1000);
30:                 ctrl.releaseWrite();
31:             }
32:         } catch (InterruptedException e){}
33:     }
34: }
```

```

35: public class ReadWriteController {
36:     private int readers = 0;
37:     private boolean writing = false;
38:     private int numWaitingWriters = 0;
39:     private boolean readersTurn = false;

40:     synchronized public void acquireRead()
41:         throws InterruptedException {
42:         while (writing || (numWaitingWriters>0 && !readersTurn)) {
43:             wait();
44:         }
45:         ++readers;
46:     }

47:     synchronized public void releaseRead() {
48:         --readers;
49:         readersTurn=false;
50:         if(readers==0) notifyAll();
51:     }

52:     synchronized public void acquireWrite()
53:         throws InterruptedException {
54:         ++numWaitingWriters;
55:         while (readersTurn || writing) { wait(); }
56:         --numWaitingWriters;
57:         writing = true;
58:     }

59:     synchronized public void releaseWrite() {
60:         writing = false;
61:         readersTurn = true;
62:         notifyAll();
63:     }
64: }

```

Answer the following subquestions. For each subquestion, **exactly one option is correct**; do not mark multiple options, and do not provide any justification or explanation.

Q6a. Suppose that there are ten Readers and ten Writers running concurrently. Which one of the following statements is *guaranteed* to be true?

- A) No thread will ever starve
- B) At most one Reader can starve but no Writer will
- C) At most one Writer can starve but no Reader will
- D) More than one Reader can starve but no Writer will
- E) More than one Writer can starve but no Reader will
- F) None of the above

[(+4,+1) marks]

Q6b. Suppose that there are ten Readers and ten Writers running concurrently. We replace `while(readersTurn || writing)` with `while(readers>0 || writing)` in `acquireWrite()` (line 55). Which one of the following statements is *guaranteed* to be true?

- A) No thread will ever starve
- B) At most one Reader can starve but no Writer will
- C) At most one Writer can starve but no Reader will
- D) More than one Reader can starve but no Writer will
- E) More than one Writer can starve but no Reader will
- F) None of the above

[(+4,+1) marks]

Q6c. Suppose that there is only one Reader and ten Writers running concurrently. We replace the `while` keyword with the `if` keyword in `acquireRead()` (line 42). Which one of the following statements is true?

- A) A deadlock can occur
- B) The only Reader can starve
- C) All the Writers can starve
- D) A Writer can write and the Reader can read at the same time
- E) Two or more of the above can occur
- F) None of the above can occur

[(+4,+1) marks]

Q6d. Suppose that we replace `notifyAll()` with `notify()` in `releaseWrite()` (line 62). Which one of the following statements is true?

- A) A deadlock can never occur
- B) A deadlock can occur if there's at least one Writer or Reader
- C) A deadlock can occur if there are at least two Writers
- D) A deadlock can occur if there are at least two Readers
- E) A deadlock can occur regardless of the number of Readers and Writers
- F) None of the above

[(+4,+1) marks]

[Total for Question 6: 16 marks]